



Licenciatura Engenharia Informática e Multimédia
Instituto Superior de Engenharia de Lisboa
Ano letivo 2022/2023

Computação Física
Relatório: Trabalho Prático 1

Turma: 22D	Grupo: 6
Nome: Miguel Alcobia	Número: 50746
Nome: Fábio Pestana	Número: 50756
Nome: Rafael Dias	Número: 50773
Professor: Jorge Pais	

Data: 28 de Março de 2023

Índice

Índice	I
Lista de figuras	II
Lista de tabelas	III
Abreviaturas e símbolos	IV
Objetivos	1
Exercício 1	2
Enunciado:	2
1. 1 - Modelo de entradas e saídas do circuito lógico do sistema	2
1. 2 - Tabela de verdade que descreve o funcionamento pretendido	2
1. 3 - Obtenção da expressão lógica simplificada por simplificação algébrica	3
1. 4 - Obtenção da expressão lógica simplificada através de mapa de Karnaugh	3
1. 5 - Desenho do circuito lógico simplificado	4
1. 6 - Situações de teste e resultados obtidos	4
Exercício 2	5
Enunciado:	5
2. 1 - Modelo de entradas e saídas do circuito lógico do sistema	5
2. 2 - Desenho do ASM que descreve o funcionamento pretendido	5
2. 3 - Desenho do Modelo de Moore-Mealey do circuito	6
2. 4 - Tabelas de verdade, mapas de Karnaugh e expressões simplificadas da FES e da FS	7
2. 5 - Desenho do circuito lógico final simplificado	10
2. 6 - Situações de teste e resultados obtidos	10
Bibliografia	11
Código Comentado	12
Exercício 1:	12
Exercício 2:	14

Lista de figuras

Figura 1 - Modelo de entradas e saídas do exercício 1	2
Figura 2 - Mapa de Karnaugh, seguindo os resultados da Tabela 1	3
Figura 3 - Desenho do circuito lógico do exercício 1	4
Figura 4 - Montagem do circuito do exercício 1	4
Figura 5 - Modelo de entradas e saídas do exercício 2	5
Figura 8 - Modelo de Moore-Mealey do circuito	7
Figura 9 - Mapa de Karnaugh de J1	8
Figura 10 - Mapa de Karnaugh de K1	8
Figura 11 - Mapa de Karnaugh de J0	8
Figura 12 - Mpa de Karnaugh de K0	8
Figura 13 - Desenho do circuito lógico final simplificado	10
Figura 14 - Montagem do circuito do excercício 2	10

Lista de tabelas

Tabela 1 - Tabela de verdade que descreve o funcionamento do sistema 3

Tabela 2 - T.V. da F.E.S. 7

Tabela 3 - Tabela de Verdade do FF JK edge-triggered..... 7

Tabela 4 - Tabela da F.S. 9

Tabela 5 - Tabela do valor de Q em JK 9

Abreviaturas e símbolos

Lista de abreviaturas

ASM	Algorithmic State Machine
F.E.S.	Função do Estado Seguinte
F.F.	Flip-Flop
F.S.	Função de Saída
T.V.	Tabela de Verdade

Lista de símbolos

$X/$ e $\bar{X}$	Negação de X (NOT)
$+$	Disjunção (OR)
$.$	Conjunção (AND)

Objetivos

Neste trabalho, os alunos tinham como objetivo desenvolver um projeto de circuitos combinatórios. Para realizar esta atividade foi necessário os alunos utilizarem funções lógicas, obtidas através de uma tabela de verdade que descreve o problema enunciado, simplificar a função lógica do problema utilizando as propriedades lógicas e confirmando esta função com os mapas de Karnaugh, desenhando em seguida o respetivo circuito lógico.

Nesta atividade, os alunos também desenvolveram um projeto de circuitos sequenciais baseados num ASM fundamentado no problema. Para este projeto foi necessário a utilização de *flip-flops* do tipo J-K Edge-Triggered.

Para ambos projetos, os alunos desenvolveram e implementaram código para o Arduino, tendo em vista simular os circuitos.

Exercício 1

Enunciado:

“Projete um sistema que emite um aviso contínuo (de luz ou de som), no caso das luzes de um automóvel serem deixadas acesas inadvertidamente. Este aviso deve ser emitido enquanto as luzes estiverem acesas, no caso de alguma das portas dianteiras do veículo estar aberta, tendo o motor desligado.”

- Carvalho, Carlos; Niehus, Manfred & Pais, Jorge. (2023). *Computação Física 1.º Trabalho Prático*

1. 1 - Modelo de entradas e saídas do circuito lógico do sistema

Para desenhar o modelo de entradas e saídas começou-se por definir quais seriam as entradas: um sensor para detetar se a luz está ligada (L); um sensor para a porta esquerda (SPE), outro para a direita (SPD) e mais um para o motor (M). A saída do aviso foi definida como S.

L quando o seu valor lógico é 0, significa que as luzes do carro estão desligadas e quando é 1, estão ligadas. **SPE** quando o seu valor lógico é 0, significa que a porta esquerda está fechada e quando é 1, está aberta. **SPD** segue o mesmo raciocínio, mas para a porta direita. **M** quando o seu valor lógico é 0, significa que o motor está desligado e quando é 1, está ligado. Por fim, **S** representa a saída pretendida e só fica a 1 quando estão reunidas as condições propostas no enunciado para o alarme disparar.

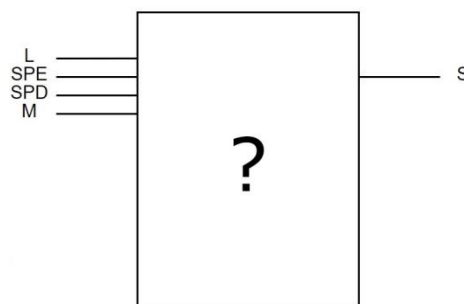


Figura 1 - Modelo de entradas e saídas do exercício 1

1. 2 - Tabela de verdade que descreve o funcionamento pretendido

Construiu-se a tabela de verdade usou-se as 4 variáveis das entradas e os valores de S foram assumidos 1 apenas quando se apresentavam as condições para o alarme disparar: A luz tem de estar ativada (L=1), pelo menos uma das portas tem de estar abertas (SPD=1 ou SPE=1) e o motor tem de estar desligado (M=0).

A tabela é apresentada em seguida:

L	SPD	SPE	M	S
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	0
0	1	0	0	0
0	1	0	1	0
0	1	1	0	0
0	1	1	1	0
1	0	0	0	0
1	0	0	1	0
1	0	1	0	1
1	0	1	1	0
1	1	0	0	1
1	1	0	1	0
1	1	1	0	1
1	1	1	1	0

Tabela 1 - Tabela de verdade que descreve o funcionamento do sistema

1.3 - Obtenção da expressão lógica simplificada por simplificação algébrica

$$\begin{aligned}
 & L.\overline{SPD}.SPE.\overline{M} + L.SP.D.\overline{SPE}.\overline{M} + L.SP.D.SPE.\overline{M} = \\
 & = L.\overline{M}.(\overline{SPD}.SPE + SPD.\overline{SPE} + SPD.SPE) = \\
 & = L.\overline{M}.(\overline{SPD}.SPE + (\overline{SPE} + SPE).SPD) = \\
 & = L.\overline{M}.(\overline{SPD}.SPE + 1.SP.D) = \\
 & = L.\overline{M}.((\overline{SPD}.SPE) + SPD) = \\
 & = L.\overline{M}.((\overline{SPD} + SPD).(SPD + SPE)) = \\
 & = L.\overline{M}.(1.(SPD + SPE)) = \\
 & = L.\overline{M}.(SPD + SPE)
 \end{aligned}$$

Inversa da propriedade distributiva
 Inversa da propriedade distributiva
 1 é elemento neutro da interseção
 Teorema da complementação
 Propriedade distributiva
 Teorema da complementação
 1 é elemento neutro da interseção

1.4 - Obtenção da expressão lógica simplificada através de mapa de Karnaugh

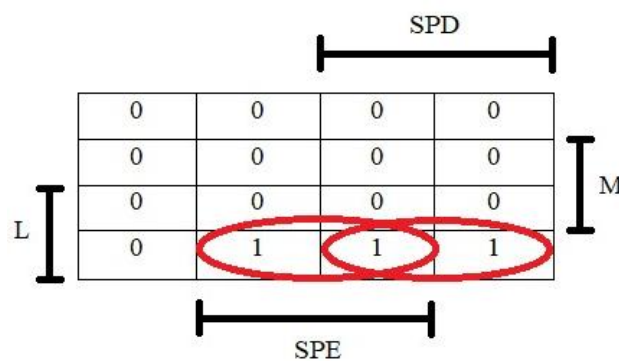


Figura 2 - Mapa de Karnaugh, seguindo os resultados da Tabela 1

Expressão obtida a partir do mapa do Karnaugh:

$$L \cdot SPE \cdot M / + L \cdot SPD \cdot M / =$$

$$= L \cdot M / \cdot (SPD + SPE)$$

Inversa da propriedade distributiva

1. 5 - Desenho do circuito lógico simplificado

Para desenhar o circuito lógico apenas se traduziu a expressão obtida no mapa de Karnaugh da Figura 2 com a ajuda das representações das portas lógicas.

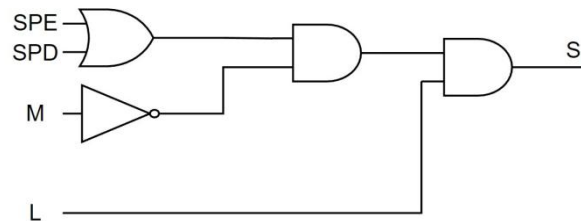


Figura 3 - Desenho do circuito lógico do exercício 1

1. 6 - Situações de teste e resultados obtidos

Fez-se a montagem ilustrada na Figura 4, para simular o problema enunciado.

O Piezo será o que efetuará o alarme. O LED D1 demonstra se as luzes estão ou não ligadas. Os botões S1 e S2 simulam as portas direita e esquerda, respectivamente. Por último, os *jumper*s simulam o motor (D7) e D8 são as luzes.

Primeiramente, atribuiu-se um valor fixo às variáveis do motor e das luzes; porém, depois do primeiro teste e a pedido do professor, alterou-se a montagem através de *jumper*s para atribuir os valores às variáveis, a partir do `digitalRead()`.

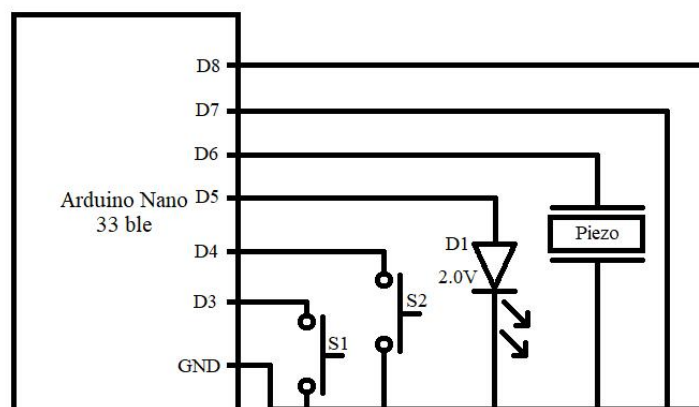


Figura 4 - Montagem do circuito do exercício 1

Exercício 2

Enunciado:

“Projete um sistema, com o mesmo propósito do descrito no problema anterior, mas que tenha a capacidade de emitir o aviso pretendido, de uma forma intermitente, ao ritmo de um sinal de clock.”

- Carvalho, Carlos; Niehus, Manfred & Pais, Jorge. (2023). *Computação Física 1.º Trabalho Prático*

2. 1 - Modelo de entradas e saídas do circuito lógico do sistema

Para desenhar o modelo de entradas e saídas começou-se por definir quais seriam as entradas: um sensor para detetar se a luz está ligada (L); um sensor para a porta esquerda (SPE), outro para a direita (SPD) e mais um para o motor (M). A saída do aviso foi definida como S.

L quando o seu valor lógico é 0, significa que as luzes do carro estão desligadas e quando é 1, estão ligadas. **SPE** quando o seu valor lógico é 0, significa que a porta esquerda está fechada e quando é 1, está aberta. **SPD** segue o mesmo raciocínio, mas para a porta direita. **M** quando o seu valor lógico é 0, significa que o motor está desligado e quando é 1, está ligado. Por fim, **S** representa a saída pretendida e só fica a 1 quando estão reunidas as condições propostas no enunciado para o alarme disparar.

Diferentemente do Exercício 1, desta vez usou-se o clock para se fazer o alarme de modo intermitente.

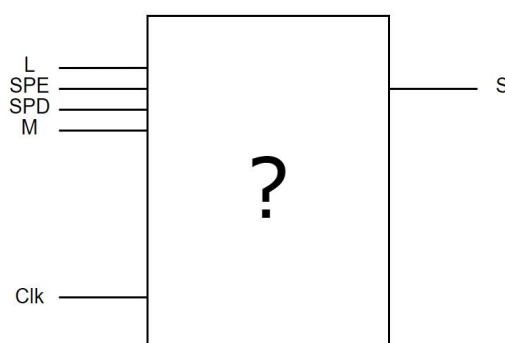


Figura 5 - Modelo de entradas e saídas do exercício 2

2. 2 - Desenho do ASM que descreve o funcionamento pretendido

Ao elaborar este ASM, pensou-se em trabalhar com quatro estados (sendo que um deles não é utilizado). O estado 00 foi batizado “Idle”, que representa o momento em que o sistema está

desligado em estado de espera. O estado 11 foi batizado “ON”, que representa o momento em o alarme do sistema é ativado. O estado 01 foi batizado “OFF”, que representa o momento em o alarme do sistema é desativado, permitindo que o alarme seja intermitente.

Por último, o estado 10 não é utilizado.

A única condição de teste que foi criada, expressa as condições necessárias para que o alarme dispare. Caso as condições sejam verificadas, existe mudança de estado; caso contrário, volta para o estado “Idle”.

Em seguida está o ASM elaborado:

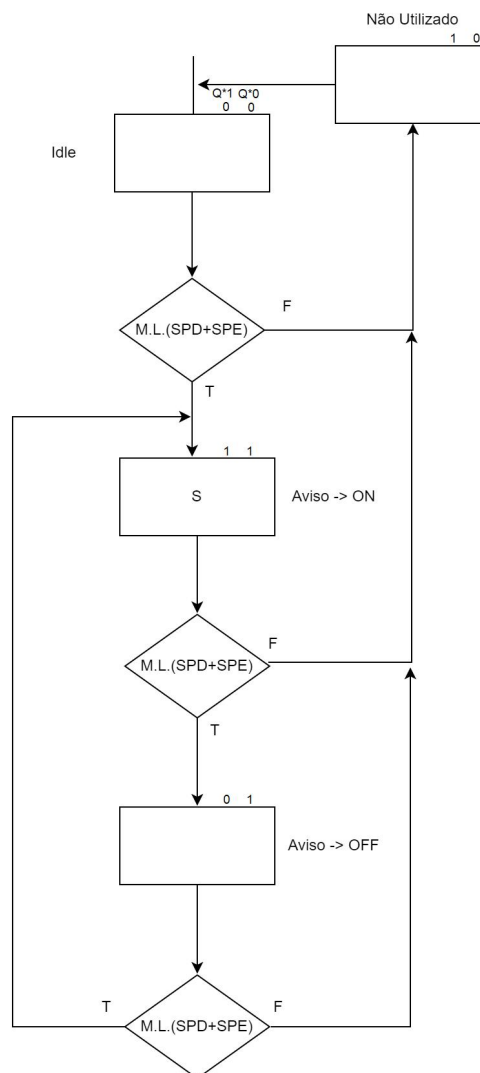


Figura 6 - Diagrama ASM do circuito

2. 3 - Desenho do Modelo de Moore-Mealey do circuito

Neste ponto do projeto, usamos o flip-flop JK edge-triggered, como pedido no enunciado para implementar Q1 e Q0. Com estas decisões tomadas, desenhou-se o seguinte modelo de Moore-Mealey:

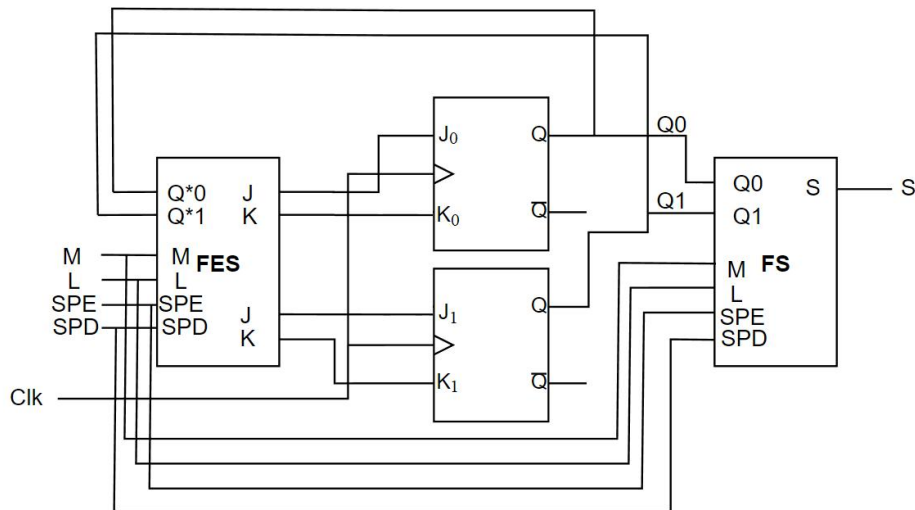


Figura 8 - Modelo de Moore-Mealey do circuito

2. 4 - Tabelas de verdade, mapas de Karnaugh e expressões simplificadas da FES e da FS

Para chegar às expressões da F.E.S., fez-se uma tabela de verdade com três variáveis e usou-se a tabela dos valores do *flip-flop* JK (Tabela 3) para chegar ao valores de J1, K1, J0, e K0. Elaborou-se também mapas de Karnaugh para os vlaores dos *flip-flops*.

Nota: Para simplificar a escrita e leitura do trabalho, a expressão lógica que apresenta as condições para ao alarme disparar ($L.\bar{M}.(SPD+SPE)$) passa a ser resumida e representada pela letra **F**.

F	Q1*	Q0*	J1	K1	J0	K0
0	0	0	0	X	0	X
0	0	1	0	X	X	1
0	1	0	X	1	0	X
0	1	1	X	1	X	1
1	0	0	1	X	1	X
1	0	1	1	X	X	0
1	1	0	X	1	0	X
1	1	1	X	1	X	0

Tabela 2 - T.V. da F.E.S.

J	K	Q
0	0	Q*
0	1	0
1	0	1
1	1	$\bar{Q}^*$

Tabela 3 - Tabela de Verdade do FF JK edge-triggered

J1:

Q^*1	X	$\bar{X}$	Q^*0
	X	X	
	0	1	
	0	1	
F			

Figura 9 - Mapa de Karnaugh de J1

K1:

Q^*1	1	1	Q^*0
	1	1	
	X	X	
	X	X	
F			

Figura 10 - Mapa de Karnaugh de K1

J0:

Q^*1	0	0	Q^*0
	X	X	
	X	X	
	0	1	
F			

Figura 11 - Mapa de Karnaugh de J0

K0:

Q^*1	X	X	Q^*0
	1	0	
	1	0	
	X	X	
F			

Figura 12 - Mapa de Karnaugh de K0

Expressões retiradas dos mapas:

$$J1 = F$$

$$K1 = 1$$

$$J0 = F \cdot Q^*1/$$

$$K0 = F/$$

Para chegar às expressões da F.S., elaborou-se uma tabela de três variáveis e recorreu-se à Tabela 3 para chegar aos valores de Q.

Estados	Q1*	Q0*	S
IDLE	0	0	0
OF	0	1	0
Não usado	1	0	0
ON	1	1	F

Tabela 4 - Tabela da F.S.

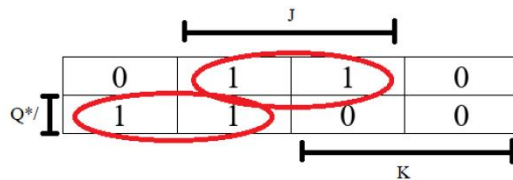
$$S = Q1*.Q0*.F$$

Expresões de JK para usar no código:

Q*	J	K	Q
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0

Tabela 5 - Tabela do valor de Q em JK

Q:



		J		
	0	1	1	0
Q*/	1	1	0	0
		K		

Expressões retiradas do mapa:

$$Q = \overline{Q}*.J + Q*.\overline{K}$$

2. 5 - Desenho do circuito lógico final simplificado

Elaborou-se o seguinte desenho do circuito lógico, já com as expressões lógicas simplificadas:

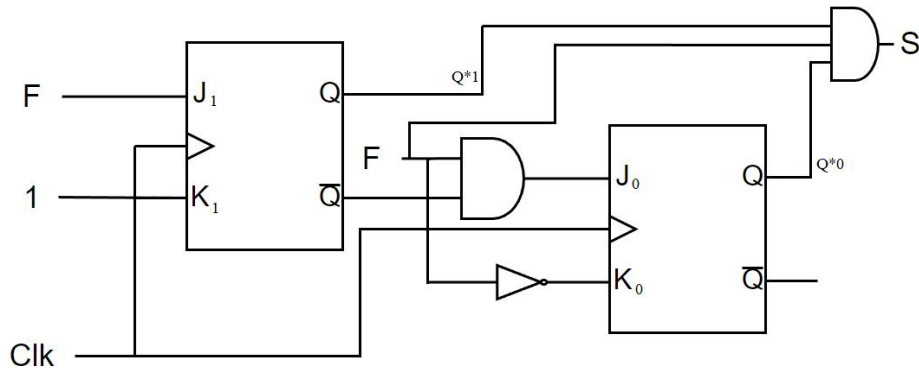


Figura 13 - Desenho do circuito lógico final simplificado

2. 6 - Situações de teste e resultados obtidos

Fez-se a montagem ilustrada na Figura 4, para simular o problema enunciado.

O Piezo será o que efetuará o alarme. O LED D1 demonstra se as luzes estão ou não ligadas. O botão S1 passou a ser utilizado para atualizar o valor do *clock* de uma forma manual, quando o botão deixa de ser pressionado (RISING); o S2 simula a porta esquerda. Por último, os *jumpers* simulam o motor (D7) e D8 são as luzes e D3 simula a porta direita .

O grupo teve alguns problemas com o funcionamento dos *pointers* na função `FlipFlopJK()` , mas com ajuda do monitor as dúvidas ficaram esclarecidas.

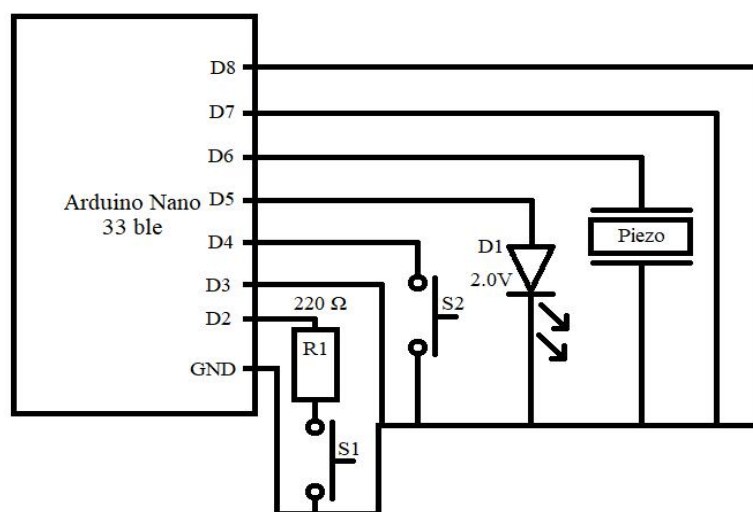


Figura 14 - Montagem do circuito do exercício 2

Bibliografia

Consulta:

Mooddle:

Pais, Jorge. (2022 - 2023). *Computação Física*

Carvalho, Carlos (2023). *Computação Física*

Desenhos:

MS Paint

Draw.io - <https://app.diagrams.net/>

Código Comentado

Exercício 1:

```
//ISEL- LEIM - Group 06 - Computação Física - TP1 - Ex 1
//Coding UTF-08

//define's

#define L 5          // pin do led - serve para demonstrar se as luzes estão ligados ou não
#define S1 3         // interruptor 1 - simula a porta direita do carro
#define S2 4         // interruptor 2 - simula a porta esquerda do carro
#define pinAviso 6   // pin do piezo - componente que demonstra a partir de um som o estado do aviso
#define M 7          // motor - simula o motor do carro
#define Lt 8         // luzes - simula as luzes do carro

//variaveis globais

bool Luz;
bool SPD;
bool SPE;
bool Aviso;
bool Motor;

void setup() {
    pinMode(L, OUTPUT);
    pinMode(S1, INPUT_PULLUP);
    pinMode(S2, INPUT_PULLUP);
    pinMode(M, INPUT_PULLUP); //apenas um jumper do pin digital 7 ao GND tendo a resistência
                              integrada do arduino que impede um curto circuito
    pinMode(Lt, INPUT_PULLUP); //apenas um jumper do pin digital 8 ao GND tendo a resistência
                              integrada do arduino que impede um curto circuito
    pinMode(pinAviso, OUTPUT);
}

void loop() {
    LerEntradas();
    CircuitoCombinatorio();
    EscreverSaidas();
}

// método do tipo void que vai ler todos os pins, utilizando o comando digitalRead(), que no setup
são definidos como INPUT_PULLUP

void LerEntradas()
{
    SPD = !digitalRead(S1); // SPD dá true/HIGH/1 quando a porta direita está aberta (S1 pressionado)
    e dá false/LOW/0 quando a porta está fechada
    SPE = !digitalRead(S2); // SPE dá true/HIGH/1 quando a porta esquerda está aberta (S2 pressionado)
    e dá false/LOW/0 quando ela está fechada
    Motor = !digitalRead(M); // Motor dá true/HIGH/1 quando o motor está ligado (jumper ligado á
    breadboard) e dá false/LOW/0 quando o motor está desligado (jumper não está ligado á breadboard)
    Luz= !digitalRead(Lt); // Luz dá true/HIGH/1 quando as luzes estão ligadas (jumper ligado á
    breadboard) e dá false/LOW/0 quando as luzes estão desligadas (jumper não está ligado á breadboard)
}
```

```

// método do tipo void que utiliza os valores obtidos no método LerEntradas() e atualiza uma
variável booleana apartir de uma expressão lógica

void CircuitoCombinatorio()
{
    Aviso = (Luz && (SPD||SPE) && !Motor);
    // expressão lógica simplificada, obtida através da tabela de verdade e mapa de Karnaugh (ver 1.2,
    1.3 e 1.4 do relatório)
}

// método do tipo void que atualiza o estado da LED e Piezo consoante o valor lógico das variáveis
booleanas (Luz e Aviso)
void EscreverSaidas()
{
    digitalWrite(L, Luz);
    digitalWrite(pinAviso, Aviso);
}

```

Exercício 2:

```
//ISEL- LEIM - Group 06 - Computação Física - TP1 - Ex 2
//Coding UTF-08

//define's
#define L 5          // pin do led - serve para demonstrar se as luzes estão ligados ou não
#define S1 3         // simula a porta direita do carro
#define S2 4         // interruptor 2 - simula a porta esquerda do carro
#define pinAviso 6   // pin do piezo - componente que demonstra a partir de um som o estado do aviso
#define M 7          // motor - simula o motor do carro
#define Lt 8         // luzes - simula as luzes do carro
#define pinoClk 2    // pin que serve para simular o clock - feito manualmente a partir do interruptor 1

//variaveis globais
bool Luz;
bool SPD;
bool SPE;
bool Aviso;
bool Motor;
bool clk;
bool S;
bool J0, K0, J1, K1; // Entradas J e K
bool Q0, Q1; // Saída Q

void setup() {
    pinMode(L, OUTPUT);
    pinMode(S1, INPUT_PULLUP); // apenas um jumper do pin digital 3 ao GND tendo a resistência integrada
do arduino que impede um curto circuito
    pinMode(S2, INPUT_PULLUP);
    pinMode(M, INPUT_PULLUP); // apenas um jumper do pin digital 7 ao GND tendo a resistência
integrada do arduino que impede um curto circuito
    pinMode(Lt, INPUT_PULLUP); // apenas um jumper do pin digital 8 ao GND tendo a resistência
integrada do arduino que impede um curto circuito
    pinMode(pinAviso, OUTPUT);
    pinMode(pinoClk, INPUT); // utilizou-se o modo INPUT em vez do INPUT_PULLUP devido ao mau
funcionamento do clock nesse modo
    attachInterrupt(digitalPinToInterrupt(pinoClk), myclock, RISING);
    interrupts();
}

void loop() {
    LerEntradas();
    CircuitoCombinatorio();
    CircuitoSequencial();
    EscreverSaidas();
}

// método do tipo void que vai ler todos os pins, utilizando o comando digitalRead(), que no setup
são definidos como INPUT_PULLUP

void LerEntradas()
{
    SPD = !digitalRead(S1); // SPD dá true/HIGH/1 quando a porta direita está aberta (jumper ligado á
breadboard) e dá false/LOW/0 quando a porta está fechada (jumper não está ligado á breadboard)
    SPE = !digitalRead(S2); // SPE dá true/HIGH/1 quando a porta esquerda está aberta (S2 pressionado)
e dá false/LOW/0 quando ela está fechada
    Motor = !digitalRead(M); // Motor dá true/HIGH/1 quando o motor está ligado (jumper ligado á
breadboard) e dá false/LOW/0 quando o motor está desligado (jumper não está ligado á breadboard)
```

```

    Luz= !digitalRead(Lt); // Luz dá true/HIGH/1 quando as luzes estão ligadas (jumper ligado á
    breadboard) e dá false/LOW/0 quando as luzes estão desligadas (jumper não está ligado á breadboard)
}

// método do tipo void que utiliza os valores obtidos no método LerEntradas() e atualiza uma
variável booleana apartir de uma expressão lógica
// são chamados os seguintes métodos FES() e FS() para assim serem atualizadas as variáveis dos
flip-flops e o Aviso

void CircuitoCombinatorio()
{
    S = (Luz && (SPD||SPE) && !Motor);
    // expressão lógica simplificada, obtida através da tabela de verdade e mapa de Karnaugh (ver 1.2,
    1.3 e 1.4 do relatório)
    FES();
    FS();
}

// método do tipo void que atualiza o estado da LED e Piezo consoante o valor lógico das variáveis
booleanas (Luz e Aviso)

void EscreverSaidas()
{
    digitalWrite(L, Luz);
    digitalWrite(pinAviso, Aviso);
}

// método do tipo void que atribui o valor true á variável clk

void myclock()
{
    clk=true;
}

// método do tipo void que verifica o estado do clk, e no caso deste ser true, o método atualiza os
valores de Q1 e Q0 apartir da função FlipFlopJK() e atribui o valor false a clk

void CircuitoSequencial()
{
    if(clk)
    {
        FlipFlopJK(J1,K1,&Q1); //Atualização do valor de Q1
        FlipFlopJK(J0,K0,&Q0); //Atualização do valor de Q0
        clk=false;
        //O operador de endereço "&" é usado para obter o endereço de memória de uma variável, neste
        caso de Q1 e Q0
    }
}

// função do tipo void que recebe como parâmetros os valores flip-flops do tipo J-K edge-triggered
(J, K, Q)

void FlipFlopJK(bool J, bool K, bool *Q) // operador unário de indireção "*" (pointer) é usado para
acessar o valor armazenado em um endereço de memória
{
    *Q = (!*Q && J) || (*Q && !K); //atualização do valor de Q usando a expressão lógica do flip-flop
}

// função do tipo void, que utiliza as expressões da função do estado seguinte (ver 2.4 do relatório)

```

```

void FES()
{
    J1= S;
    K1= 1;
    J0= S && (!Q1);
    K0= (!S);
}

```

// função do tipo void, que utiliza as expressões da função de saída (ver 2.4 do relatório)

```

void FS()
{
    Aviso = Q1 && Q0 && S;
}

```